ENME 441

07 – Concurrency

`_thread` / `asyncio` / `interrupts`

Topics

Three different to run multiple codes concurrently.



PART 1

_thread

Preemptive multithreading

- Low-level threading module
- Runs on real OS threads
- True parallel scheduling



PART 2

asyncio

Cooperative multitasking

- Single-threaded event loop
- Tasks yield with await
- Works on nearly every port



PART 3

Interrupts

Hardware-triggered code

- Fires on a pin or timer
- Runs outside normal flow
- Hands off with schedule()

Why Concurrency?

- **Blink an LED, turn a motor, poll a sensor, monitor communications, etc., each with different timing constraints**

  No job should block others from making progress or interfere with precise timing of events.

- **Handle network traffic without freezing input**

  Waiting on Wi-Fi communication shouldn't stall other code.

- **Three different solutions:**
  _thread → OS-level threads (OS controls task switching: preemptive multitasking)
  asyncio → cooperative tasks (our code controls task switching: cooperative multitasking)
  interrupts → react to hardware directly (hardware-level events trigger a thread controlled by the OS)

- **No true parallel processing on the ESP32-C3**

  The C3 has a single-core CPU, so concurrent tasks are interleaved rather than truly simultaneous.

01

_thread

Preemptive multitasking



_thread: Preemptive Multitasking

`_thread` mirrors CPython's low-level `threading` module.

1

Real OS threads

Functions run on preemptive threads

2

Scheduler can interrupt anytime

The interpreter may switch threads between any two bytecode instructions; our code has no control over this timing.

3

Shared memory model requires locking

All threads share the same global state, so concurrent writes need explicit locking.

4

Threads are interleaved, not truly parallel

MicroPython runs all threads on the ESP32-C3's single core and holds a global lock, so threads never execute simultaneously. Instead, the system preemptively time-slices between them — it can switch threads at any moment, without your code yielding.

Basic Usage: _thread

_thread.start_new_thread() launches a given function on a new thread

```
import _thread
import time

def worker(calling, delay):
    for i in range(5):
        time.sleep(delay)
        print(f"{calling} count: {i}")

# Start worker() in a new thread:
_thread.start_new_thread(worker, ("thread",1))

# Call worker() from main thread concurrently:
worker("main", 0.5)
```

- **start_new_thread(fn, args)**

  Spawns fn(*args) in a new thread; returns immediately so current thread continues to run at the same time.

- **Both run concurrently**

  "main" and ”worker" run at the same time, interleaved with timing controlled by the OS.

- **No return value**

  Threads cannot return a value. Instead use a shared list, queue, or lock to get results back.

Synchronizing shared variables with Locks

Sharing variables across threads needs explicit protection: every read-modify-write cycle needs a lock

```
import _thread
import time

# Create a lock object:
lock = _thread.allocate_lock()

counter = 0
start = time.ticks_ms()

def increment():
    global counter
    for _ in range(100000):
        with lock:
            counter += 1

_thread.start_new_thread(increment, ())
increment()
print(f'count={counter}, time={time.ticks_diff(time.ticks_ms(), start)}')
```

- **allocate_lock()**

  Creates a lock object with acquire() / release().

- **Without lock**

  The two threads reading-then-writing the counter can lose updates (classic “race condition”).

- **`with lock`**

  The context manager tries to acquire the lock. If the lock is checked out by another thread, the current thread waits until the lock is available. Multiple locks can be used to protect different variables within the code.

- **Try it**

  See what happens when the lock is omitted – the counter likely will not reach the correct value (200000) after both threads run.

Locks add delays

Code takes longer to run when using locks since (a) tasks must pause while another task has the lock checked out, and (b) the context manager adds CPU overhead.

Sharing Multiple Locked Values in a List

A list is the simplest shared buffer.

```
import _thread
shared = []
lock = _thread.allocate_lock()

def put():
    for i in range(10):
        with lock:
            shared.append(i)
def get():
    while True:
        with lock:
            if shared:
                item = shared.pop(0)
                print("got", item)
        time.sleep_ms(5)
_thread.start_new_thread(put, ())
get()
```

- **Queue example**

  This code is a basic thread-safe FIFO queue in a producer/consumer pattern: put() appends to the end, get() takes from the front with pop(0), and the lock makes sure only one thread touches the list at a time.

- **Check-then-act stays inside the lock**

  Test "if shared:" before popping as one locked step, or two get() calls at different points in the code can race for the last item.

- **time.sleep_ms(5)**

  A short sleep is needed to prevent get() from hogging the lock

ESP32 Threads Are Real RTOS Tasks

Arbitrary number of threads can run concurrently (interleaved, with preemptive time-slicing to switch between them)

```
import _thread
import time

def count(name, period):
    while True:
        print(name)
        time.sleep(period)

# each call returns immediately and leaves
# a new task running in the background
_thread.start_new_thread(count, ("fast", 0.2))
_thread.start_new_thread(count, (" "*5+"medium", 0.5))
_thread.start_new_thread(count, (" "*13+"slow", 1.0))

count(" "*18+"glacial", 2.0)   # main thread works too
```

- **Each thread is a FreeRTOS task**

  start_new_thread() hands the function to the ESP-IDF scheduler, which time-slices it against every other task.

- **Thread count is only limited by RAM**

  Every task carries its own stack; call _thread.stack_size(n) before spawning if a specific thread needs more.

- **An IRQ will commandeer the currently-running thread**

  Remember: if an IRQ occurs, the callback function will pause the current thread for execution, and only return control after running to completion.

OPTIONAL

Building a Thread-Safe Queue

Use a queue to wrap a list and lock together. MicroPython does not include a queue module, so create a custom queue class instead.

```
import _thread
import time

class Queue:
    def __init__(self):
        self.items = []      # underlying list of queued items
        self.lock = _thread.allocate_lock()  # lock to guard self.items

    def put(self, item):
        """Add an item to the queue"""
        with self.lock:    # acquire the lock
            self.items.append(item)

    def get(self):
        """Block until an item is available, then remove and
           return it (FIFO order)."""
        while True:    # keep polling until something is in the queue
            with self.lock:
                if self.items:    # if at least one item is waiting
                    return self.items.pop(0)
            time.sleep_ms(5)      # brief sleep to avoid hogging lock

q = Queue()
_thread.start_new_thread(lambda: q.put("hello"), ())
print(q.get())
```

- **Using a queue simplified thread communication**

  A queue manages data sequentially while providing coordinated communication between multiple threads. It allows multiple threads to share data without causing a race condition.

- **The queue owns the lock**

  Callers never touch the lock directly — the class hides it, so locking can't be forgotten elsewhere in the code.

OPTIONAL

Using the Queue – Single `put()` and `get()` Calls

```
q = Queue() # create a shared queue instance

# Start a new thread that puts "hello" into the queue.
# The lambda wraps q.put("hello") so it can be passed as
# the thread's entry function. The empty tuple () is the
# (empty) argument list for that function.
_thread.start_new_thread(lambda: q.put("hello"), ())

# Main thread calls get(), which blocks (polling every 5 ms) until
# the second thread has put "hello" into the queue. Only then
# does the print statement execute.
print(q.get())
```

- **put() adds an item to the queue**

  The lock in put() ensures that the list is not modified elsewhere in the code while the current put() call is executing.

- **get() returns the earliest item added to the queue (FIFO)**

  get() uses polling with a lock rather than a blocking wait — it repeatedly checks "is there anything here?" and sleeps 5ms between checks if not.

OPTIONAL

Using the Queue – Multiple put() and get() calls

```
q = Queue()

def producer():
    """Run on a separate thread. Puts 3 items into the queue,
    one every 100ms, then exits. Nothing is consuming yet,
    so items just pile up in q.items in the order added."""
    for i in range(3):
        msg = f"item-{i}"
        print("putting:", msg)
        q.put(msg)
        time.sleep_ms(100)

# Start the producer thread. It begins putting items right away.
_thread.start_new_thread(producer, ())

# Main thread waits until the producer is done, so all 3 puts
# happen before any get() is called.
time.sleep_ms(400)

# Drain the queue — FIFO order: item-0, item-1, item-2
for _ in range(3):
    print("got:", q.get())
```

- **Multiple put() and/or get() calls can be made in sequence**

  The size of the queue (self.list) can grow or shrink on demand.

_thread: Things to Watch For

No join()

There's no built-in way to wait for a thread to finish — signal completion yourself with a lock or flag.

Locks are your responsibility

Any global or shared object written from more than one thread needs a lock around the write.

Limited, fixed stack size

Use _thread.stack_size() to raise it before spawning if a thread needs deep recursion.

Port support varies

Not every board ships _thread — some limit you to a single extra thread, others omit it entirely.

02

asyncio

Cooperative multitasking



asyncio: Cooperative Multitasking

`asyncio` (uasyncio) runs multiple tasks on a single thread. Unlike threads, in which transitions between tasks are scheduled by the interpreter, asyncio requires transition points to be defined within the code itself.

1

One event loop, multiple tasks

No OS-level parallelism → no data races.

2

Tasks yield at `await` points

A task keeps running until it hits an `await` — then, and only then, can another task run.

3

Lightweight

Tasks are far cheaper than threads since no per-task stack allocation.

4

Good fit for I/O-bound work

Waiting on sensors, timers, sockets, serial ports, etc. — one loop runs something while another loop waits.

Async Coroutines

`async def` defines a coroutine function (a kind of generator)

```
import asyncio

async def read_sensor():
    print("reading")
    await asyncio.sleep(0.1)
    return 42

# asyncio.run() runs the coroutine in a new asyncio task loop
val = asyncio.run(read_sensor())
print(val)          # 42

async def main():
    # schedule the coroutine on the currently running event loop:
    t = asyncio.create_task(read_sensor())
    print(await t)  # 42

# earlier asyncio.run() call has returned so asyncio loop has ended.
# need to restart with another asyncio.run() before
# asyncio.create_task() within main()
asyncio.run(main())
```

- **`async def` defines a coroutine function**

  Calling the coroutine directly returns a generator object and executes nothing. The function code does not run until something drives the object (see below).

- **`asyncio.run()`**

  Creates the asyncio task loop and runs the given coroutine as the first task in the loop
  - The asyncio loop ends when the function called by run() ends.

- **`create_task()`**

  Add a task to an existing loop started with asyncio.run().
  - `t = asyncio.create_task(f)` wraps coroutine f in a task and queues the task by adding it to the asyncio task loop.

- **`await`**

  Suspends the current coroutine (task) and returns control to the asyncio loop, allowing other queued tasks to run.
  - The asyncio loop runs one task at a time, each until its next `await` (or task completion).
  - await is only legal inside an async def coroutine.

await: Transferring Control Between Tasks

`await t` does two jobs at once:

1. pause the main code until task `t` ends, and
2. let the asyncio loop run other queued tasks while waiting.

```python
async def grab(name, t):
    await asyncio.sleep(t)
    print(name, "done")
    return t

async def main():
    # await *without* registering tasks: sequential - 3 seconds total
    await grab("a", 1)
    await grab("b", 2)

    # await with registered tasks: concurrent - 2 seconds total
    ta = asyncio.create_task(grab("a", 1))
    tb = asyncio.create_task(grab("b", 2))
    await ta
    await tb

asyncio.run(main())    # start the asyncio task loop and run main()
```

- **await means suspend current task, run others'**

  Control goes back to the asyncio loop, which runs the next task in the loop queue. Only async-aware objects can be awaited (never a blocking function).

- **`await` without registered tasks run sequentially**

  Awaiting a coroutine runs it to completion before the next line. 1 s then 2 s (3 s total sleep time).

- **create_task() adds a task to the loop queue**

  create_task() registers the task with the loop, allowing multiple tasks to run concurrently on the await statement (2 s total sleep time)

await: Run Sequence

time	what happens when `asyncio.run(main())` reaches the two `create_task()` calls
0.0 s	`ta = asyncio.create_task(grab("a", 1))` — task ta is created and added to the asyncio loop queue. `grab`'s body has not run.
	`tb = asyncio.create_task(grab("b", 2))` — task tb is created and added to the asyncio loop queue (after ta).
	`await ta` — ta is not done, so **main suspends** and returns control to the asyncio loop (containing ta then tb)
	`await asyncio.sleep` — The first queued task (ta) runs, hits `await asyncio.sleep(1)` inside `grab()`, and suspends, returning control to **main**.
	`await tb` — tb is not done, so so **main suspends** and returns control to the asyncio loop
	`await asyncio.sleep` — The 2nd queued task (tb) runs, hits `await asyncio.sleep(2)` inside `grab()`, and suspends, returning control to **main**. No additional tasks are in the queue, so the loop idles.
1.0 s	`ta`'s 1 s sleep expires; ta prints and returns. **Completing ta returns control to main** which was suspended after the initial `await ta` line.
	`main` resumes and executes `await tb`. tb is not done (tb has been running concurrently with ta for 1 s) so main suspends again.
2.0 s	`tb` finishes, so `main` resumes and the code ends. **Total elapsed: 2 s.**

asyncio Use Example

Use `asyncio` to blink two LEDs at different rates, concurrently

```
import asyncio
from machine import Pin

led1 = Pin(0, Pin.OUT, value=0)
led2 = Pin(1, Pin.OUT, value=1)

async def blink(p, delay):
    while True:
        p.toggle()
        await asyncio.sleep(delay)

async def main():
    asyncio.create_task(blink(led1, 0.5))
    asyncio.create_task(blink(led2, 0.3))
    await asyncio.sleep(5)

asyncio.run(main())
```

- **await asyncio.sleep(5)**

  This statement yields control to the async loop for 5 seconds, allowing the queued blink tasks to run.

  After 5 s control is returned to main(), which returns. At that point the asyncio.run() statement returns and the async loop ends.

  Outstanding tasks remaining in the asyncio loop are abandoned/garbage-collected. In the example here, this means the blink tasks stop running, and the overall code terminates since there are no further statements.

Never Block the Loop

await only works with functions that have an async counterpart. In general, this means blocking functions like timers and sockets (e.g. time.sleep → asyncio.sleep).

```python
import asyncio, time

async def bad():
    while True:
        time.sleep(1)         # freezes ALL tasks
        print("tick")

async def worse():
    while True:
        await time.sleep(1)   # TypeError -
        print("tick")         # and it blocked
                              # BEFORE failing

async def good():
    while True:
        await asyncio.sleep(1)    # yields
        print("tick")
```

- **await cannot rescue a blocking call**

  time.sleep(1) runs to completion first, freezing every task; only then does await see its None return value and raise TypeError.

- **Blocking needs an async counterpart**

  asyncio.sleep() returns an object describing the wait, so the loop can schedule around it. A function with no such version simply cannot be awaited.

- **Long compute → blocking drivers**

  A for loop with no await, a blocking socket read, a while pin.value(): spin , etc. can stall everything. Break the code up with await asyncio.sleep(0) statements, or use a _thread.

OPTIONAL

Useful asyncio Patterns

`asyncio.gather(t1, t2)`	Run several coroutines concurrently and wait for all of them to finish.
`asyncio.wait_for(coro, timeout)`	Add a timeout to any awaitable; raises on expiry.
`StreamReader / StreamWriter`	Non-blocking sockets for network protocols without manual polling.
`ThreadSafeFlag`	Safely signal an asyncio task from an ISR or a _thread thread.
`asyncio.sleep_ms(ms)`	Millisecond-resolution sleep — avoids float overhead on tight loops.

03

Interrupt requests (IRQs)



Hardware-triggered execution

Interrupts: Hardware-Triggered Code

The MCU, rather than your code, decides when your handler runs based on an interrupt request (IRQ)

1

Fires on a hardware event

A pin changing state, but also: a timer expiring, a UART byte arriving, etc.

2

Runs outside your normal call stack

An ISR preempts _thread threads and the asyncio loop alike; neither expects code to run between their instructions.

3

Extremely constrained context

No heap allocation on some ports, no blocking calls, no printing in tight loops — get in, record the event, get out.

4

The surest way to miss an event

Spend too long in a handler and you can miss the next interrupt, or starve everything else on the chip.

Basic Usage: `irq()`

`Pin.irq()` calls the handler function when a GPIO pin changes state

```
from machine import Pin

def handler(pin):
    print("edge on", pin)

button = Pin(0, Pin.IN, Pin.PULL_UP)
button.irq(trigger=Pin.IRQ_FALLING, handler=handler)

while True:
    pass
```

- **Pin.irq(trigger, handler)**

  Registers handler(pin) to run on the given edge — falling, rising, or both.

- **The handler receives the pin**

  Useful when one handler serves several pins; check which one fired.

- **Multiple edges at once**

  trigger=Pin.IRQ_FALLING | Pin.IRQ_RISING catches both edges with one registration.

IRQs via Hardware Timers: `Timer()`

`Timer` fires an IRQ on a defined schedule instead of a pin edge

```
from machine import Timer, Pin

led = Pin(2, Pin.OUT)
tick = 0

def on_timer(t):
    global tick
    tick += 1
    led.toggle()

timer = Timer(0)
timer.init(period=500, mode=Timer.PERIODIC, callback=on_timer)

# timer.deinit() to free a timer and stop the callback.
```

- **Hardware vs. sottware timers**

  Timer(0) and Timer(1) are the available hardware timers on the ESP32-C3.
  Timer(-1) is a software timer that is less precise.

- **PERIODIC vs ONE_SHOT**

  mode=Timer.PERIODIC repeats every period ms;
  Timer.ONE_SHOT fires once and stops.

- **Still an ISR**

  A timer callback runs in interrupt context exactly like Pin.irq() — keep it tiny and defer real work with schedule().

Getting Data Out Safely: `schedule()`

`micropython.schedule()` defers real work to normal context

```
import micropython
from machine import Pin

micropython.alloc_emergency_exception_buf(100)
count = 0

def deferred(pin_arg):
    global count
    count += 1
    print("button presses:", count)

def handler(pin):
    micropython.schedule(deferred, pin)

button = Pin(0, Pin.IN, Pin.PULL_UP)
button.irq(trigger=Pin.IRQ_FALLING, handler=handler)
```

- **handler() stays tiny**

  The handler calls micropython.schedule() with deferred() as a callback function to be run in the main execution loop ”very soon”.

- **Key avantage of schedule()**

  The callback function passed to schedule() is not limited by the restricted context of the IRQ (where the heap is locked and memory allocation is forbidden)

- **alloc_emergency_exception_buf()**

  Lets MicroPython report exceptions raised inside an ISR, which otherwise fail silently.

Interrupts: Things to Watch For

Keep handlers tiny

Time spent in an ISR is time every other interrupt, thread, and asyncio task has to wait on.

Defer real work using schedule()

Treat the handler as a trigger, not a place to do the actual work — hand work back to the main thread asap.

No locks, no blocking calls

_thread.allocate_lock() and time.sleep() can deadlock or simply aren't safe from interrupt context.

Avoid allocation

Many ports raise on heap allocation inside an ISR — build buffers ahead of time, don't create new objects in the handler.

Comparison

Aspect	_thread	asyncio	Interrupts
Scheduling	Preemptive (OS-level)	Cooperative (event loop)	Hardware-triggered
Memory use	High: per-thread stack	Low: tasks share one stack	Minimal: no stack of its own
Best for	Blocking calls you can't rewrite	I/O-bound waiting, many tasks	Instant response to hardware
Shared state	Needs explicit locks	Usually lock-free (one thread)	Never lock — defer with schedule()
Port support	ESP32, Pyboard, Unix port…	Nearly universal	Universal — core MCU feature

LAB ASSIGNMENT

Lab 7: Concurrency

